```
// COS30008 - Problem Set 3, 2025

#pragma once

#include <ostream>

template<typename K, typename V>
class SortablePair {
public:
    // construct from key & value
    SortablePair(const K &aFirst = K{}, const V &
aSecond = V{}) noexcept
        : fFirst(aFirst), fSecond(aSecond) {
    }

    // Access the key
    const K &first() const noexcept { return fFirst; }

    // Access the value
    const V &second() const noexcept { return fSecond
; }

    // True if both key and value match
    bool operator==(const SortablePair &aOther) const
noexcept {
        return fFirst == aOther.fFirst
               && fSecond == aOther.fSecond;
    }

    // Define “less” by higher key first
    bool operator<(const SortablePair &aOther) const
noexcept {
        return fFirst > aOther.fFirst;
    }

    // Stream output as (key,value)
    friend std::ostream &operator<<(std::ostream &os,
const SortablePair &p) {
        return os << '(' << p.fFirst << ',' << p.
fSecond << ')';
    }
```

```
private:
    K fFirst; // priority
    V fSecond; // associated value
};
```